

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #12

Graph isomorphism

Agenda

- **Graphs isomorphism**
 - with metaprogramming
 - various implementations (compare & contrast analysis)

Graphs isomorphism problem statement

- Let us:
 - $G1=(V1,E1)$
 - $G2=(V2,E2)$
 - $G1$ iso $G2$ iff
 - i) $|V1|=|V2|$
 - ii) $\exists \varphi : V1 \rightarrow V2$ a bijection so that
 - iii) $\forall x, y \in V1 [(x, y) \in E1 \Leftrightarrow (\varphi(x), \varphi(y)) \in E2]$
- In simpler words:
 - Graphs are essentially "the same" in structure, differing only by the names or arrangement of their vertices.
 - φ matches up vertices one-to-one while preserving adjacency
- Ex: Are the 2 graphs isomorphs?

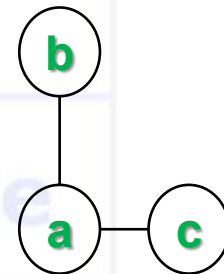
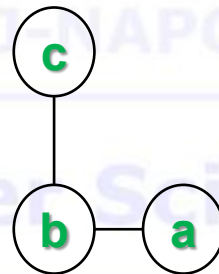
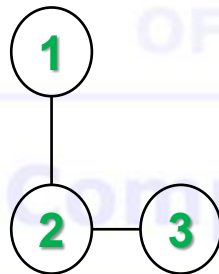
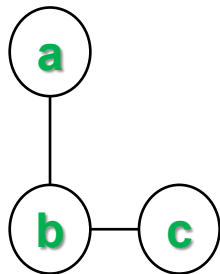
```
graph1([
  n(a,[b,c,d]),
  n(b,[a,c]),
  n(c,[a,b,d]),
  n(d,[a,c])
]).
```

```
graph2([
  n(1,[2,4]),
  n(2,[1,3,4]),
  n(3,[2,4]),
  n(4,[1,2,3])
]).
```

Graphs isomorphism

Simple example

- Consider these two graphs:
 - Graph A** has vertices labelled $\{a,b,c\}$ with edges $(a,b),(b,c)$.
 - Graph B** has vertices labelled $\{1,2,3\}$ with edges $(1,2),(2,3)$.



Graphs isomorphism approach (v1)

```
?- graph1(G1), graph2(G2), iso_graph1(G1,G2).
```



```
iso_graph1(L1,L2):- eq_perm(L1,L2,eq_neighb).
```



```
eq_perm([H1|T1],L2,EQ):- % is the perm predicate with an equivalence
    delete(H2,L2,T2), % nondeterministic behavior. Implemented how?
    P=..[EQ,H1,H2], % which is created here (metapredicate)
    call(P), % and called here to execute
    eq_perm(T1,T2,EQ).
```



```
eq_perm([],[],_). % succeeds iff sets have the same cardinality
% an immediate improvement would check cardinalities beforehand
```



```
eq_neighb(n(N1,L1),n(N2,L2)):- % 2 neighb pairs are equivalent
    eq_node(N1,N2), % if the nodes are equivalent
    eq_perm(L1,L2,eq_node). % and the neighb lists are equivalent
```



```
delete(H,[H|T],T). % what if we add a cut here?
delete(X,[H|T],[H|R]):-
    delete(X,T,R). % what if we add the [] clause?
```

Nodes equivalence

(implements φ function)

V1 – default logic with side effects

- p a dynamic predicate, used for the nonmonotonic reasoning
- implements φ function (to form p as akb)

```
eq_node(N1,N2):-           % if nodes N1 and N2 already form a pair in the akb
    p(N1,N2).              % the evaluation continues
eq_node(N1,_):-            % if N1 forms a pair with some OTHER node
    p(N1,_),!,             % we get an inconsistency, so should NOT allow
    fail.                  % (N1,N2) form a pair, so, fail to backtrack
eq_node(_,N2):-            % symmetric on N2
    p(_,N2),!,             % not an injective function
    fail.                  % How/where bijection is insured?
eq_node(N1,N2):-           % if you reach here, there is no inconsistency
    asserta(p(N1,N2)).      % hence pair N1, N2 is legitimate.
eq_node(N1,N2):-           % if at a later moment,
    retract(p(N1,N2)),!,    % although pair N1, N2 is legitimate,
                             % the reasoning cannot conclude,
                             % remove it from the akb, and
                             % fail to backtrack and
                             % resume execution WITHOUT the pair in the akb
    fail.
```

Graphs isomorphism approach (v1)

```
?- graph1(G1), graph2(G2), iso_graph1(G1,G2).
```

```
iso_graph1(L1,L2):- eq_perm(L1,L2,eq_neighb).
```

```
eq_perm([H1|T1],L2,EQ):-  
    delete(H2,L2,T2),  
    P=..[EQ,H1,H2],  
    call(P),  
    eq_perm(T1,T2,EQ).
```

```
eq_perm([],[],_).
```

```
eq_neighb(n(N1,L1),n(N2,L2)):-  
    eq_node(N1,N2),  
    eq_perm(L1,L2,eq_node).
```

```
delete(H,[H|T],T).  
delete(X,[H|T],[H|R]):- delete(X,T,R).
```

```
eq_node(N1,N2):- p(N1,N2).  
eq_node(N1,_):- p(N1,_), !, fail.  
eq_node(_,N2):- p(_,N2), !, fail.  
eq_node(N1,N2):- asserta(p(N1,N2)).  
eq_node(N1,N2):- retract(p(N1,N2)), !, fail.
```

Nodes equivalence

V2 – with arguments (lists)

```
?- graph1(G1), graph2(G2), iso_graph2(G1,G2).
% instead of the akb p in v1, here we store the pairs in the arg list
% arg 4 = partial list, empty initially (arg to model the akb)
% arg 5 = final list; a copy of the partial list at the end of the execution
% (arg with the status of the akb at the end; the bijection)
iso_graph2(L1,L2):- eq_perm(L1,L2,eq_neighb,[],Lout).

eq_perm([H1|T1],L2,EQ,LI,LO):- % same as in v1
    delete(H2,L2,T2),
    P=..[EQ,H1,H2,LI,Lint], % same but with args
    call(P),
    eq_perm(T1,T2,EQ,Lint,LO).
eq_perm([],[],_,L,L).

eq_neighb(n(N1,L1),n(N2,L2),LI,LO):-
    eq_node(N1,N2,LI,Lint),
    eq_perm(L1,L2,eq_node,Lint,LO).
```


Nodes equivalence

(implements φ function)

V2 – with arguments (lists)

- p is a pair of nodes in the list argument
- models φ function

```
eq_node(N1,N2,LI,LI):-  
    member(p(N1,N2),LI), !.  
eq_node(N1,N2,LI,[p(N1,N2)|LI]):-  
    not(member(p(N1,_),LI)),  
    not(member(p(_,N2),LI)).
```

- These 2 clauses do the same as those 5 in v1

Nodes equivalence

(φ function v2 VS v1)

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), !.  
  
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
  
    not (member (p (N1, _), LI)),  
  
    not (member (p (_, N2), LI)).
```

```
eq_node (N1, N2) :-  
    p (N1, N2) .  
eq_node (N1, _) :-  
    p (N1, _), !,  
    fail.  
eq_node (_, N2) :-  
    p (_, N2), !,  
    fail.  
eq_node (N1, N2) :-  
    asserta (p (N1, N2)) .  
eq_node (N1, N2) :-  
    retract (p (N1, N2)), !,  
    fail.
```

Nodes equivalence

(implements φ function)

V3 – with arguments (incomplete lists)

- Same as v2 but lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the “not found” case
- Just the equivalence changes
 - (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node(N1,N2,[p(N1,N2)|_]) :- !.
```

```
eq_node(N1,_,[p(N1,_)|_]) :- !,  
    fail.
```

```
eq_node(_,N2,[p(_,N2)|_]) :- !,  
    fail.
```

```
eq_node(N1,N2,[_|T]) :-  
    eq_node(N1,N2,T).
```

- Compare v3 with v1. Why is clause 1 (from v1) missing (in v3)?
- Compare v3 with v2.

Nodes equivalence

(ϕ function v3 VS v1)

```
eq_node (N1, N2, [p(N1, N2) | _]) :- !.  
eq_node (N1, _, [p(N1, _) | _]) :-  
    !,  
    fail.  
eq_node (_, N2, [p(_, N2) | _]) :-  
    !,  
    fail.  
eq_node (N1, N2, [_ | T]) :-  
    eq_node (N1, N2, T).
```

```
eq_node (N1, N2) :-  
    p(N1, N2).  
eq_node (N1, _) :-  
    p(N1, _), !,  
    fail.  
eq_node (_, N2) :-  
    p(_, N2), !,  
    fail.  
eq_node (N1, N2) :-  
    asserta(p(N1, N2)).  
eq_node (N1, N2) :-  
    retract(p(N1, N2)), !,  
    fail.
```

Nodes equivalence

(φ function v3 VS v2)

```
eq_node(N1,N2,[p(N1,N2)|_]) :- !.  
eq_node(N1,_, [p(N1,_) | _]) :-  
    !,  
    fail.  
eq_node(_,N2,[p(_,N2)|_]) :-  
    !,  
    fail.  
eq_node(N1,N2,[_|T]) :-  
    eq_node(N1,N2,T).
```

```
eq_node(N1,N2,LI,LI) :-  
    member(p(N1,N2),LI), !.  
eq_node(N1,N2,LI,[p(N1,N2)|LI]) :-  
    not(member(p(N1,_) ,LI)),  
  
    not(member(p(_,N2) ,LI)).
```

Nodes equivalence

(implements φ function)

V4 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the “not found” case
- Just the equivalence changes
 - (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node4 (N1,N2,LI) :-  
    member_il (p (N1,N2) , LI) , !.  
eq_node4 (N1,N2, [p (N1,N2) | LI]) :-  
    not (member_il (p (N1,_), LI)) ,  
    not (member_il (p (_,N2), LI)) .
```

```
member_il (_, L) :- var (L) , ! , fail.  
member_il (X, [X|_]) :- !.  
member_il (X, [_|T]) :- member_il (X, T).
```

- Compare v4 with v2.

Nodes equivalence

(φ function v4 VS v2)

```
eq_node (N1, N2, LI) :-  
    member_il (p (N1, N2), LI), !.
```

```
eq_node (N1, N2, [p (N1, N2) | LI]) :-  
  
    not (member_il (p (N1, _), LI)),  
  
    not (member_il (p (_, N2), LI)).
```

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), !.
```

```
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
  
    not (member (p (N1, _), LI)),  
  
    not (member (p (_, N2), LI)).
```